

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 3 – Experiments (5 QUESTIONS)

Course Code:	MLA0407	Course Title:	Deep Learning
CO Assessed:	CO1	Assessment:	Assessment Tool 3 – Experiments
Weightage:	35% of CIA	Total Marks:	(5 Questions × 10 Marks)
CO1 Statement:	<i>Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.</i>		
Student Name:	MAHALAKSHMI.P	Reg. No.:	192365066
Section / Batch:	-	Date:	23.07.26
Sign:	MAHALAKSHMI.P	Date:	23.07.26

Code of Conduct

I MAHALAKSHMI.P(192365066) (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: MAHALAKSHMI.P

QUESTIONS: 5

Total Marks: 50

Annexure A

Experiments

Duration: 1hr

```

X = [[0,0],[0,1],[1,0],[1,1]]
Y = [0,0,0,1]
w1 = 0
w2 = 0
b = 0
lr = 0.1
for epoch in range(10):
    for i in range(len(X)):
        x1, x2 = X[i]
        net = x1*w1 + x2*w2 + b
        if net >= 0:
            pred = 1
        else:
            pred = 0
        error = Y[i] - pred
        w1 = w1 + lr * error * x1
        w2 = w2 + lr * error * x2
        b = b + lr * error
print("Final Weights:")
print("w1 =", w1)
print("w2 =", w2)
print("Bias =", b)
print("\nPredictions:")
correct = 0
for i in range(len(X)):
    x1, x2 = X[i]
    net = x1*w1 + x2*w2 + b

    if net >= 0:
        pred = 1
    else:
        pred = 0
    print(X[i], "->", pred)

    if pred == Y[i]:
        correct += 1
accuracy = (correct / len(X)) * 100
print("\nAccuracy =", accuracy, "%")

```

```
x = 10
learning_rate = 0.1
iterations = 20

print("Iteration\tCost")

for i in range(iterations):
    cost = x * x
    print(i + 1, "\t\t", round(cost, 4))

    gradient = 2 * x
    x = x - learning_rate * gradient

print("\nFinal x =", round(x, 4))
```

Perceptron for XOR Gate

```
X = [[0,0],[0,1],[1,0],[1,1]]  
Y = [0,1,1,0]
```

```
w1 = 0  
w2 = 0  
b = 0
```

```
lr = 0.1
```

```
for epoch in range(10):  
    for i in range(len(X)):  
        x1, x2 = X[i]  
        net = x1*w1 + x2*w2 + b  
  
        if net >= 0:  
            pred = 1  
        else:  
            pred = 0  
  
        error = Y[i] - pred  
  
        w1 = w1 + lr * error * x1  
        w2 = w2 + lr * error * x2  
        b = b + lr * error
```

```
print("Predictions:")  
for i in range(len(X)):  
    x1, x2 = X[i]  
    net = x1*w1 + x2*w2 + b  
    pred = 1 if net >= 0 else 0  
    print(X[i], "->", pred)
```

```
print("\nXOR cannot be classified by a single-layer perceptron.")  
print("Alternative: Multi-Layer Perceptron (MLP).")
```

```

# Perceptron for OR Gate
X = [[0,0],[0,1],[1,0],[1,1]]
Y = [0,1,1,1]
w1 = 0
w2 = 0
b = 0
lr = 0.1
for epoch in range(10):
    for i in range(len(X)):
        x1, x2 = X[i]
        net = x1*w1 + x2*w2 + b

        if net >= 0:
            pred = 1
        else:
            pred = 0

        error = Y[i] - pred

        w1 = w1 + lr * error * x1
        w2 = w2 + lr * error * x2
        b = b + lr * error

print("Final Weights:")
print("w1 =", w1)
print("w2 =", w2)
print("Bias =", b)

print("\nPredictions:")
for i in range(len(X)):
    x1, x2 = X[i]
    net = x1*w1 + x2*w2 + b
    pred = 1 if net >= 0 else 0
    print(X[i], "->", pred)

```



```
def step(x):  
    return 1 if x >= 0 else 0  
  
weights = [0.5, 0.5]  
bias = -0.7  
  
inputs = [  
    [0, 0],  
    [0, 1],  
    [1, 0],  
    [1, 1]  
]  
  
for x in inputs:  
    net = x[0]*weights[0] + x[1]*weights[1] + bias  
    output = step(net)  
    print("Input:", x, "Output:", output)
```